Technical Assessment – Digital Product Passport (DPP)

Overview

The objective of this technical assessment is to evaluate your ability to design and implement a production-ready Full Stack application using modern Python web technologies.

You are required to build a **Digital Product Passport (DPP)** platform that allows companies to create, manage, and publish digital passports for their products.

Each published product must generate a unique QR Code that enables end users to access a public Product Passport page.

The assessment will evaluate your software architecture, backend development, frontend implementation, database design, security practices, testing, and code quality.

Required Technology Stack

Backend

- Python 3.12+
- FastAPI
- SQLAlchemy 2.x (ORM)
- Alembic (Database Migrations)
- PostgreSQL
- JWT Authentication
- Pydantic v2
- OpenAPI / Swagger (FastAPI built-in)
- Pytest

Frontend

- React
- Next.js
- TypeScript
- Material UI (or equivalent)

Deployment

- Docker
- Docker Compose

Functional Requirements

1. Authentication

Implement user authentication using email and password.

Roles

- Administrator
- Editor

Users must authenticate using JWT Bearer Tokens.

---

2. Dashboard

The dashboard should display:

- Total Products
- Published Product Passports
- Generated QR Codes
- Total Passport Views

---

3. Product Management

Implement complete CRUD functionality.

Each product must include the following information.

Field	Type
Product Name	Text
SKU	Text
Serial Number	Text
Category	Select
Description	Text Area
Production Date	Date
Country of Origin	Select
Status	Draft / Published

---

4. Materials

Each product may contain multiple materials.

Field
Material Name

Field

Percentage

Country of Origin

Recyclable

---

5. Sustainability

Each product should include sustainability information.

Field

Carbon Footprint

Water Consumption

Recycled Material (%)

Repairability Score

Recyclable

---

6. Certifications

Support multiple certifications per product.

Field

Certification Name

Issuing Authority

Issue Date

Expiration Date

PDF Document

---

7. Documents

Allow uploading documents such as:

- User Manual
- Warranty
- Technical Datasheet

---

8. Images

Allow uploading:

- Cover Image
- Product Gallery

---

9. QR Code Generation

When a product is published, the application must automatically generate:

- a unique UUID
- a QR Code image
- a public Product Passport URL

Example

/passport/{uuid}

The QR Code should redirect users to the public Product Passport page.

Public Product Passport

The public page must be fully responsive and accessible without authentication.

Header

Display:

- Brand Logo
- Product Image
- Product Name
- SKU
- Serial Number
- "Verified Product" badge

Product Information

Display:

- Description
- Category
- Production Date
- Country of Origin

Materials

Display a table containing:

- Material
- Percentage
- Origin

Certifications

Display all certifications with downloadable PDF files.

Sustainability

Display the following information using cards or similar UI components:

- Carbon Footprint
- Water Consumption
- Recycled Material
- Repairability Score

Documents

Allow downloading:

- User Manual
- Warranty
- Technical Datasheet

Passport Information

Display:

- Passport ID
- Creation Date
- Version
- Status
- Verification Status

QR Code Tracking

Each time the QR Code is scanned, record the following information:

- Timestamp
- IP Address
- Browser
- Operating System
- Browser Language
- Country (mock implementation is acceptable)

Analytics

Create an analytics dashboard showing:

- QR Code scans today
- Weekly scans
- Most viewed products
- Latest scans

Frontend Requirements

Create a modern and responsive Back Office.

Navigation menu:

- Dashboard
- Products
- Product Passports
- Analytics
- Users
- Settings

Product List

Display the following columns:

- Product Image
- Product Name
- SKU
- Status
- QR Code
- Total Views

- Actions

Available actions:

- View
- Edit
- Delete
- Open Passport
- Download QR Code

Product Editor

The product editor should be organized into tabs.

General Information

Basic product information.

Materials

Material management.

Sustainability

Environmental information.

Certifications

Certification management.

Documents

Document uploads.

Images

Image uploads.

Preview

Display the Product Passport exactly as it will appear to the end user.

Required API Endpoints

POST /auth/login

GET /products

POST /products

GET /products/{id}

PATCH /products/{id}

DELETE /products/{id}

POST /products/{id}/publish

GET /passport/{uuid}

GET /analytics

GET /dashboard

Non-Functional Requirements

The solution should demonstrate:

- Clean Architecture
- Domain-driven modular organization
- SOLID principles
- Proper exception handling
- Input validation using Pydantic
- Secure authentication
- REST API best practices
- Database normalization
- Type hints throughout the Python codebase
- Asynchronous programming where appropriate
- Comprehensive logging

Bonus Features

The following features are optional but will be considered a plus:

- Full-text product search
- Product Passport versioning
- Soft delete
- Audit logs
- Redis caching
- Pagination and advanced filtering
- Export Product Passport as PDF

- Drag & Drop uploads
- Background tasks using Celery or FastAPI BackgroundTasks
- Automated tests (unit, integration, end-to-end)
- Object storage integration (Amazon S3 or MinIO)

Recommended Project Structure

Backend

backend/

|

├── app/

| ├── api/

| ├── core/

| ├── auth/

| ├── users/

| ├── products/

| ├── materials/

| ├── sustainability/

| ├── certifications/

| ├── documents/

| ├── analytics/

| ├── passport/

| ├── dashboard/

| ├── database/

| └── schemas/

|

├── alembic/

├── tests/

├── Dockerfile

└── requirements.txt

Frontend

frontend/

```
|
├── app/
├── components/
├── hooks/
├── services/
├── types/
├── public/
└── Dockerfile
```

Deliverables

Please provide:

- Complete source code
- Backend and Frontend projects
- Docker Compose configuration
- Database migrations (Alembic)
- Seed scripts
- OpenAPI / Swagger documentation
- README with installation and execution instructions
- Unit and integration tests
- A short architecture document (2–3 pages) describing:
    - System architecture
    - Design decisions
    - Database design
    - Security approach
    - Scalability considerations
    - Performance optimization strategies
    - Possible future improvements
